

Cost Monitoring & Optimization in AI Infrastructure

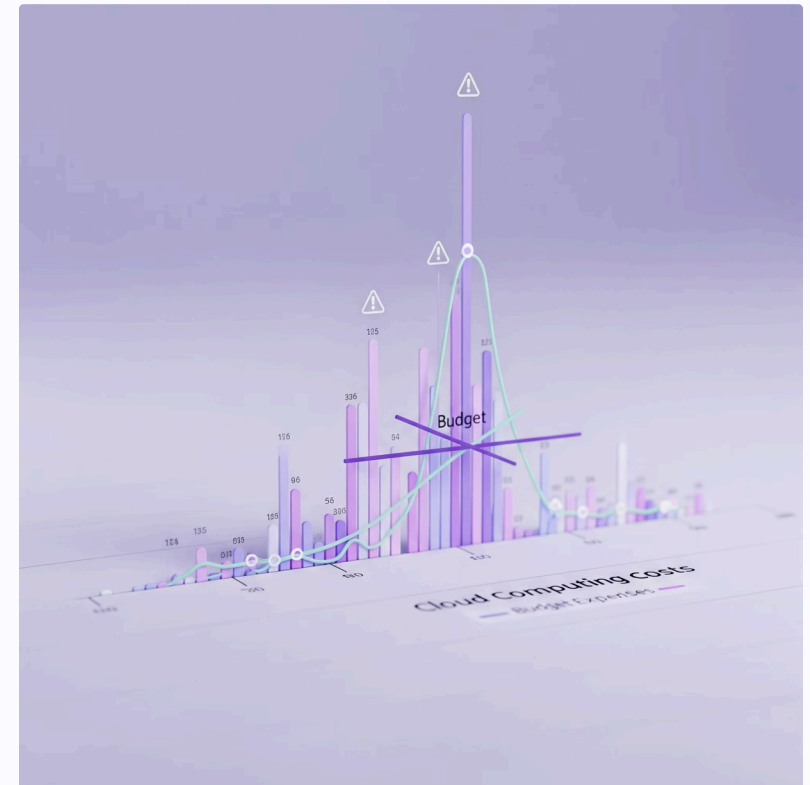
Uncontrolled AI infrastructure costs can quickly spiral out of control. This presentation explores how to effectively **observe, attribute, and optimize** these expenses without compromising your service level objectives.



Why Cost Monitoring Matters

The financial impact of poorly configured AI systems can be severe:

- Cloud bills can **double overnight** with inefficient configurations
- Budget constraints directly affect **Capstone project feasibility**
- Visibility enables better infrastructure decisions
- Cost optimization is a critical component of **MLOps maturity**



The Four Cost Dimensions in AI Infrastructure

Compute

GPUs/TPUs per hour - typically the largest expense in AI workloads

Storage

Datasets, model checkpoints, logs, and feature stores - costs accumulate over time

Networking

Data transfer between services, API calls, and cloud egress fees

Ops/Overhead

Monitoring infrastructure, CI/CD pipelines, model retraining, and idle resources

Each dimension requires specific monitoring and optimization strategies to control costs effectively.

FinOps Principles for AI Infrastructure



Visibility

Understand what you're spending and why - detailed resource tracking

Accountability

Tag resources by team, environment, and project for clear ownership

Optimization

Tune workloads to achieve SLOs with minimal resource consumption

Forecasting

Plan GPU/TPU usage against budget constraints and project timelines

Tagging & Attribution: The Foundation

Effective tagging is the cornerstone of cost visibility and accountability:

Resource Tagging Strategy

Tag every resource consistently:

- team=nlp
- env=prod
- capstone_id=vision-01

Dashboard Integration

Use consistent labels to power FinOps dashboards and reports

Multi-tenant Attribution

Enable detailed **cost-per-team** and **cost-per-feature** reporting



Monitoring Compute Spend

Export GPU/CPU hours to cost metrics for comprehensive visibility:

Key Metrics to Track

- **\$/training run** - total cost of model development
- **\$/inference (per 1k requests)** - serving cost efficiency
- **GPU utilization %** - identify idle waste

Recommended Tools

Cloud-native billing (AWS CUR, GCP Billing Export), Prometheus for metrics, Grafana for visualization



Monitoring Storage Costs



Track Usage Patterns

Monitor by bucket, storage tier, and retention policy



Key Storage Metrics

- TB stored per dataset/model
- Lifecycle tier transitions
- **\$/TB-month** trend line



Alert Configuration

Set up alerts for unexpected growth (e.g., logs filling S3 buckets)



Networking Cost Controls

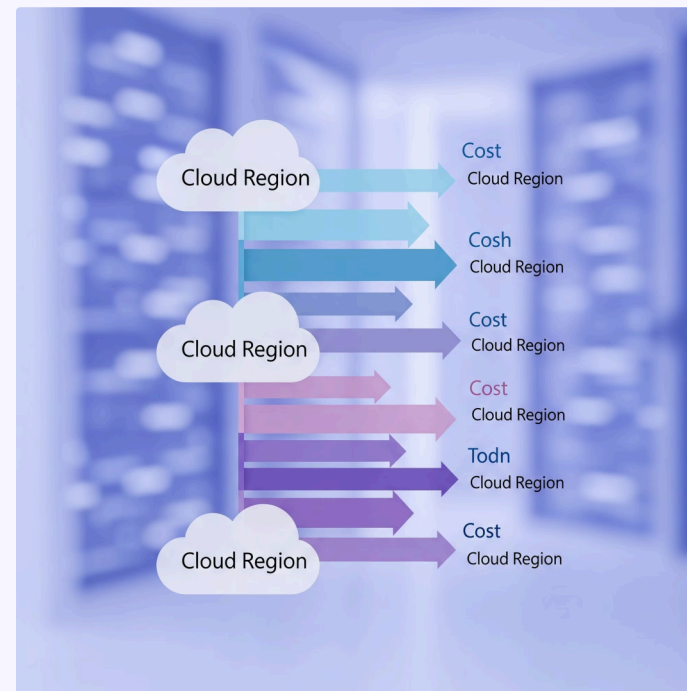
Cloud egress is the silent budget killer

Critical Metrics

- GB transferred per service/region
- CDN cache hit/miss ratio
- Cross-region data movement

Strategic Approach

Co-locate compute and data whenever possible, and avoid cross-region transfers to minimize egress charges



Essential Dashboards to Build



Compute Dashboard

- GPU hours consumed
- \$/1k inference requests
- Resource idle percentage



Storage Dashboard

- TB trend over time
- Hot vs. cold storage ratio
- 30-day forecast



Network Dashboard

- GB transfer per API
- Egress cost by destination
- Transfer optimization opportunities



Total Spend Dashboard

- Costs by project/team
- Drilldowns: service → namespace → pod
- Budget vs. actual tracking

Implement these in Grafana with appropriate drill-down capabilities for detailed analysis.

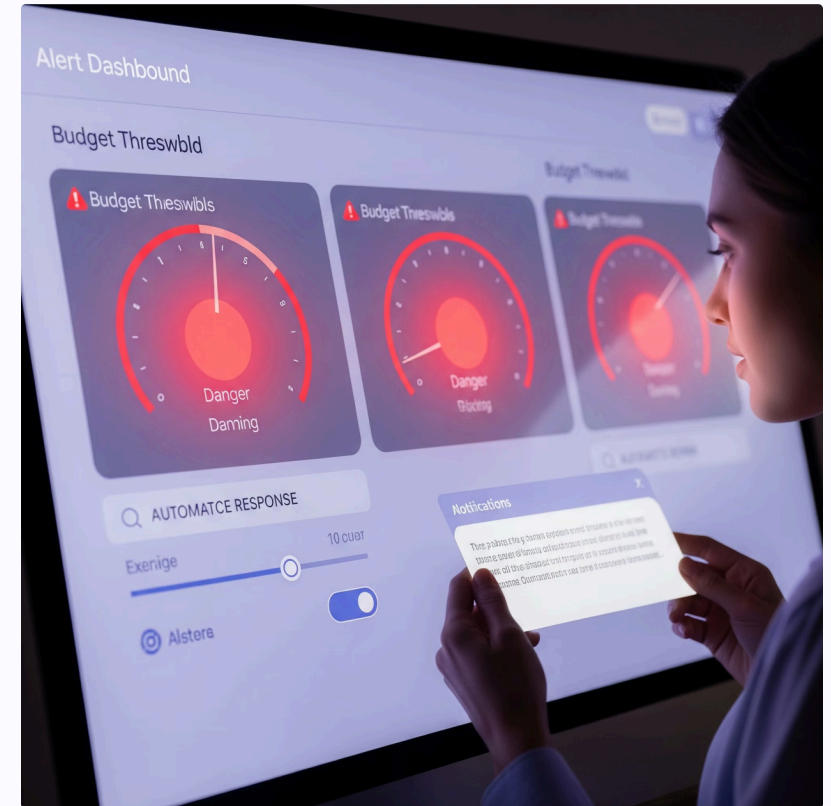
Setting Up Alerts & Budgets

Budget Guardrails

- **Critical Alert:** >80% budget consumed
- **Warning Ticket:** Cost spikes >20% above daily baseline

Automated Controls

- Auto-shutdown for idle clusters after defined threshold
- Quotas to limit maximum GPU nodes per team
- Approval workflows for high-cost resources



Optimization Strategies: Compute



Spot/Preemptible GPUs

Use for training workloads with frequent checkpointing to save 60-90% on costs



Mixed Precision Training

Implement Automatic Mixed Precision (AMP) to reduce memory usage and increase throughput



Model Compression

Apply pruning, distillation, and quantization techniques to reduce compute requirements



Inference Optimization

Implement autoscaling for inference pods and right-size Kubernetes requests/limits

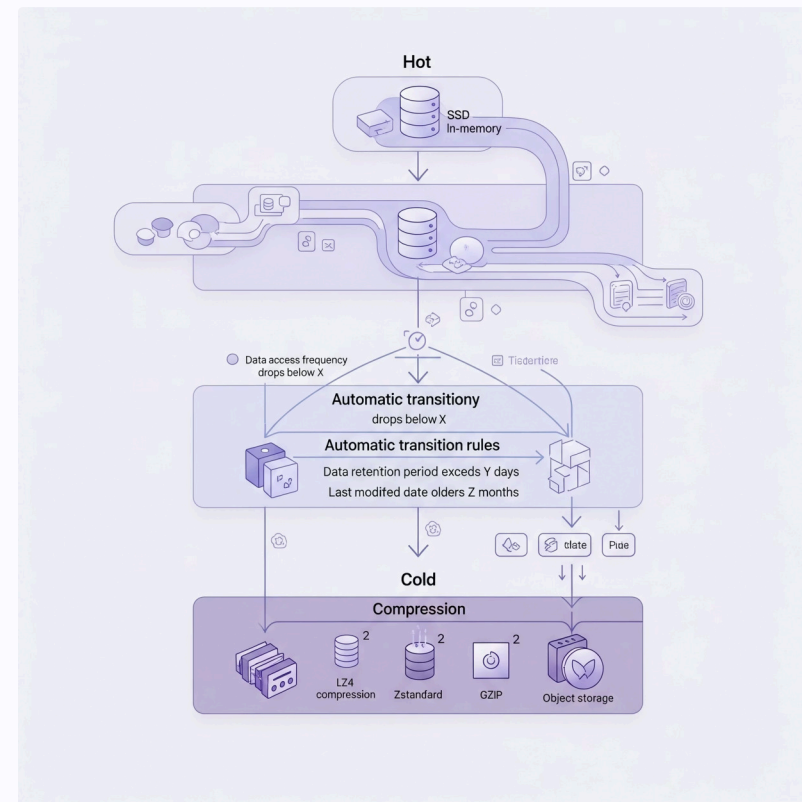
Optimization Strategies: Storage & Network

Storage Optimization

- Compress and partition data (Parquet + ZSTD)
- Archive old checkpoints, logs, and raw data
- Implement **object lifecycle rules** for automatic tiering

Network Optimization

- Cache inference responses where appropriate
- Utilize CDNs for frequently accessed outputs
- Batch API requests to reduce overhead



Case Study: Startup NLP Service

The Challenge

GPU idle time was consuming approximately **40% of the budget** with minimal workload

The Solution

Implemented Horizontal Pod Autoscaling (HPA) + job queue auto-scaling to match resource allocation with actual demand

The Results

65% cost reduction with no negative impact on service latency

Key Lesson

Utilization metrics are more important than raw GPU count when optimizing ML infrastructure costs



 Camera Footage



Downsampled Footage



Case Study: Vision Training Pipeline



The Problem

Storing all raw camera frames in hot storage led to excessive costs



The Solution

Implemented downsample + archive process after 7 days of capture



The Result

Monthly storage cost reduced from **\$3,000** to **\$700**

Lesson: Storage lifecycle rules are silent savers that continuously optimize costs without ongoing maintenance

Tools for Effective Cost Management

Cloud-Native Tools

- AWS Budgets
- GCP Cost Anomaly Detection
- Azure Cost Management

Third-Party Solutions

- Kubecost
- CloudZero
- Finout

In-Cluster Monitoring

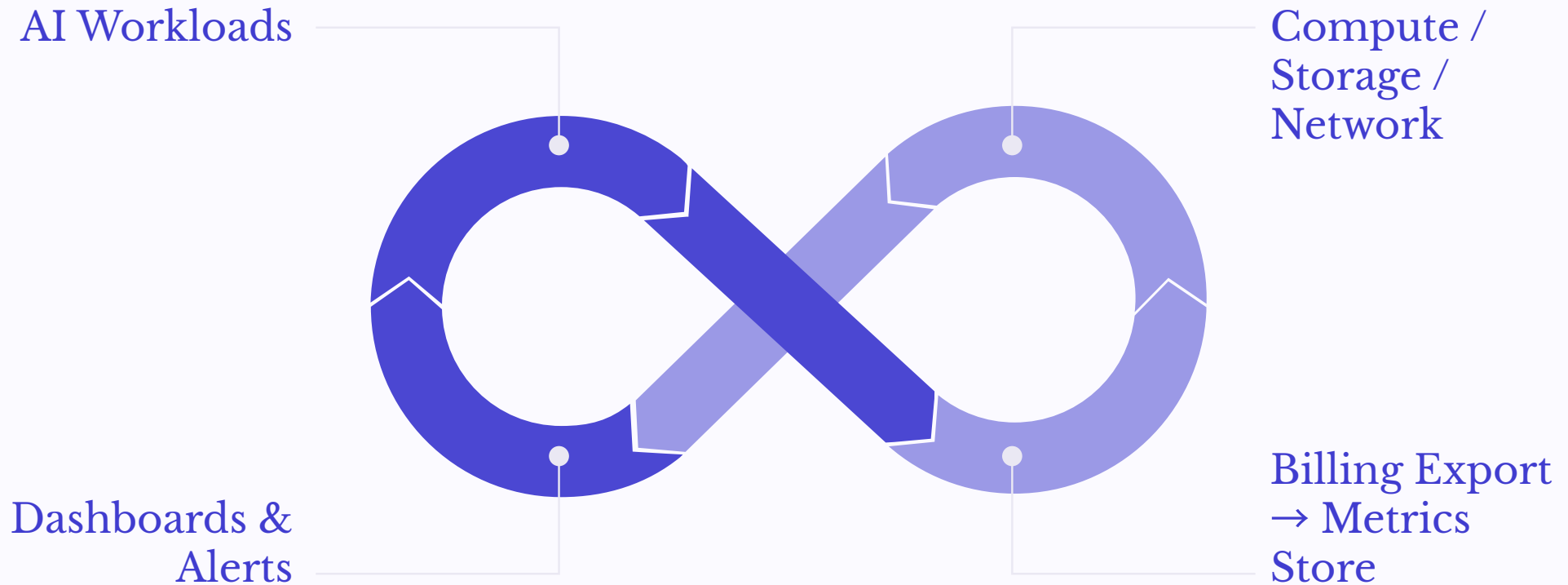
- Kubecost + Prometheus
- Grafana dashboards
- Custom metrics pipelines

Custom Solutions

- Prometheus exporters
- \$/GPU-hour calculators
- Team-specific dashboards



The Cost Observability Cycle



"Cost observability = metrics, dashboards, budgets, actions."

This continuous cycle ensures that cost data influences infrastructure decisions, driving ongoing optimization.

Key Lessons for AI Infrastructure Cost Control

1 Comprehensive Cost View

While compute is typically the heaviest expense, storage and network costs can accumulate significantly over time

2 Visibility First

Dashboards and metrics are your first line of defense
- you can't optimize what you can't measure

3 Maximum Impact Optimizations

Spot/preemptible instances combined with model optimization techniques yield the biggest savings

4 Implement Guardrails

Budgets, quotas, and auto-shutdowns prevent unexpected cost explosions

Discussion: Capstone Optimization Strategy

For your Capstone, would you first optimize GPU utilization or storage lifecycle? Which drives bigger savings in your project?

GPU Utilization Factors

- Training frequency and duration
- Inference load patterns
- Current utilization metrics
- Potential for batch processing

Storage Lifecycle Factors

- Dataset size and growth rate
- Model checkpoint frequency
- Log retention requirements
- Access patterns over time

